

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..36

RECURRING TASKS

 Copy as Markdown

Recurring tasks

A recurring task runs the same future call again and again on a schedule, such as a nightly cleanup or an hourly sync. Schedule one with `callRecurring`, choosing either a fixed interval or a `cron` expression. Serverpod schedules the next run automatically, so you set it up once.

Define the future call the same way as any other, then schedule it with `callRecurring`. Give recurring calls an `identifier` so you can stop them later.

```
import 'package:serverpod/serverpod.dart';

class ExampleFutureCall extends FutureCall {
  Future<void> cleanup(Session session) async {
    session.log('Running the recurring cleanup.');
```

Repeat on a fixed interval

Use `every` to run the call once per interval. The first run happens one interval from now, unless you pass a `start` time.

```
await pod.futureCalls
  .callRecurring(identifier: 'nightly-cleanup')
  .every(const Duration(hours: 24))
  .example
  .cleanup();
```

Pass `start` to set when the first run happens. Later runs follow one interval apart from it:

```
await pod.futureCalls
  .callRecurring(identifier: 'nightly-cleanup')
```



Ask AI

```
.every(const Duration(hours: 24), start: DateTime.now().toUtc())  
.example  
.cleanup();
```

A `start` in the past runs the call at the next scan, then continues on the interval from there. To pin a task to a wall-clock time such as 02:00 every day, use a cron expression instead.

If the server is down when a run is due, that run happens once when the server is back, then the schedule jumps ahead to the next interval boundary. The intervals missed while the server was down do not pile up and fire all at once.

Repeat on a cron schedule

Use `cron` for calendar-based schedules that a fixed interval cannot express, such as "every day at 02:00" or "every Monday". A cron expression is a short text format of five fields, `minute hour day-of-month month day-of-week`, where `*` means "every". Serverpod also accepts a sixth field, written first, for seconds.

```
await pod.futureCalls  
.callRecurring(identifier: 'daily-report')  
.cron('0 2 * * *') // Every day at 02:00 UTC.  
.example  
.cleanup();
```

Cron times are interpreted in UTC, so `'0 2 * * *'` runs at 02:00 UTC, not in the server's local time. The `Cron` parser is available from `package:serverpod/serverpod.dart` if you want to work with expressions directly. An invalid expression throws a `CronFormatException` when you schedule the call.

Stop a recurring task

A recurring task keeps rescheduling until you cancel it by its identifier:

```
await pod.futureCalls.cancel('nightly-cleanup');
```

This is why recurring calls should always be given an identifier when scheduled.

Failure behavior

Serverpod schedules the next run before it invokes the current one, so a recurring task keeps going even if one run throws. A failed run is logged and skipped, and the schedule continues. This is the opposite of a one-off call, which is not retried after a failure.

Reschedule manually

For an interval that changes from run to run, schedule the next call from inside the current one instead of using `callRecurring`. The call reschedules itself with whatever delay you compute:

```
class ExampleFutureCall extends FutureCall {
  Future<void> poll(Session session, int backoffSeconds) async {
    await _poll(session);

    // Schedule the next run with a delay computed from this run.
    await session.serverpod.futureCalls
      .callWithDelay(Duration(seconds: backoffSeconds * 2))
      .example
      .poll(backoffSeconds * 2);
  }

  Future<void> _poll(Session session) async {
    // ... the work ...
  }
}
```

Because this pattern reschedules from inside the method, a run that throws before it schedules the next one stops the loop. Prefer `callRecurring` for fixed schedules.

Related

- [Future calls](#): defining a call and scheduling one-off runs.
- [Configuration](#): how often the server checks for due calls.
- [Server roles](#): which roles run scheduled work.